

人工智能与计算机学院

School of Artificial Intelligence & Computer Science

Project-6 Report:

Digital Clock with Temperature Display

COURSE TITLE : Curriculum Design for Embedded System & Interface Technology

STUDENT NAME : KHAN MD SHAKIL

STUDENT NUMBER : 2330130220

SUPERVISOR : 袁佳祺

ACADEMIC YEAR :2025-2026

OBJECTIVE **3**

EQUIPMENT AND MATERIALS **3**

THEORY **4**

1. Real-Time Clock (RTC) 4
2. Temperature Sensing 4
3. Integration of RTC and ADC 4

PROCEDURE **5**

Step 1: Initialization 5
Step 2: Timer Configuration 5
Step 3: ADC to Temperature Conversion 5
Step 4: Real-Time Clock Display 5
Step 5: UART Debugging 5
Step 6: Testing and Validation 6

CODE IMPLEMENTATION **6**

Key Configurations 6
Main Logic in main.c 7

OBSERVATIONS AND RESULTS **9**

Observations 9

RESULTS **9**

CHALLENGES AND SOLUTIONS **10**

1. Timer Drift 10
2. Sensor Calibration 10
3. UART Noise 10

CONCLUSION **10**

Key Learnings: 10
Future Improvements: 11

REFERENCES **11**

Objective

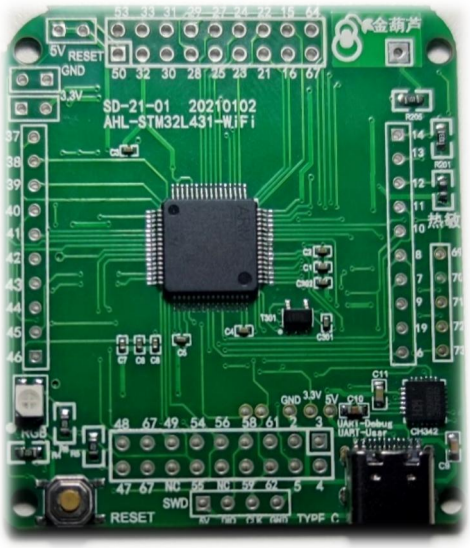
The purpose of this project is therefore to develop a **Digital Clock with Temperature Display** system using the **STM32L431RC** microcontroller. The system incorporates a **real time clock (RTC)** which aids in real time display and an on-board **temperature sensor** to display the current temperature. The readings are therefore taken at every second thereby providing an accurate display of both time and temperature.

The project showcases the best practices of microcontroller usage of **Timers, ADC, GPIO**, and **UART** peripherals. The STM32 timer module offers a one second interrupt for clock updating, while the ADC reads sensor data and computes the present temperature. This system is designed to incorporate two important features in one single embedded system; time keeping and temperature recording.

This project can be used in many fields namely; **environmental monitoring**, **climate control systems** and **automation of homes**. It gives real time information which can be enhanced for additional features such as data storage, remote server connectivity and temperature alert.

This implementation is indicative of the applicability of embedded systems in addressing common issues in the world today as well as serving as a useful learning experience for peripheral management, data collection, and system synchronization.

Equipment and Materials



- **Microcontroller**: STM32L431RC
- **Temperature Sensor**: Onboard temperature sensor
- **RTC Module:** Integrated RTC of STM32 MCU
- **Development Environment**: AHL-GEC-IDE
- **Communication Protocol**: UART for debugging
- **Additional Components**:
    - Breadboard and jumper wires for prototyping
    - USB cable for programming and UART communication

Theory

The **Digital Clock with Temperature Display** operates on two major principles: Features include the **Real Time Clock (RTC)**, and **temperature sensing**.

1. Real-Time Clock (RTC)

The RTC module is one of the basic modules of the STM32L431RC microcontroller and is integrated into the chip. It shows correct time and date to the level of **hours, minutes, seconds, day,** and **year**. The system relies on the **timer interrupt** to increase the clock each second thus providing accurate time keeping

2. Temperature Sensing

The measured temperature in the device is detected using an analog temperature sensor which provides the voltage corresponding to the temperature. The **ADC** of the microcontroller takes in this voltage and converts it into a digital value and then using a regression formula the temperature is determined. The formula is derived from the sensor's datasheet and can be represented as:

$$T = \frac{1}{\left(\frac{\log\left(\frac{R}{R_0}\right)}{B} + \frac{1}{T_0}\right)} - 273.15$$

Where:

- RRR is the resistance of the sensor
- R0R_0R0 is the reference resistance
- BBB is the beta coefficient of the thermistor

3. Integration of RTC and ADC

The design incorporates both the RTC and ADC to achieve maximum coordination in the system. The timer interrupts occur every second to refresh the clock display, as

well as to read the temperature.

Procedure

The project was implemented in the following steps:

Step 1: Initialization

- **System Clock Configuration**: The microcontroller clock was set for timing to ensure that the system was functioning as expected.
- **GPIO Setup**: Pins were set for usage for UART communication.
- **ADC Configuration**: The ADC was setup to sample the temperature sensor input with a view of acquiring its analog data.
- **RTC Initialization**: The RTC was intended to serve as a time keeping device.

Step 2: Timer Configuration

To create **one second delay** a timer interrupt was set to trigger. This makes a surety that the time as well as temperature values are updated at every single interval.

Step 3: ADC to Temperature Conversion

The raw ADC values were then put through the regression formula to obtain the corresponding temperature values. It was calibrated using the datasheet of the sensor to ascertain the best possible results.

Step 4: Real-Time Clock Display

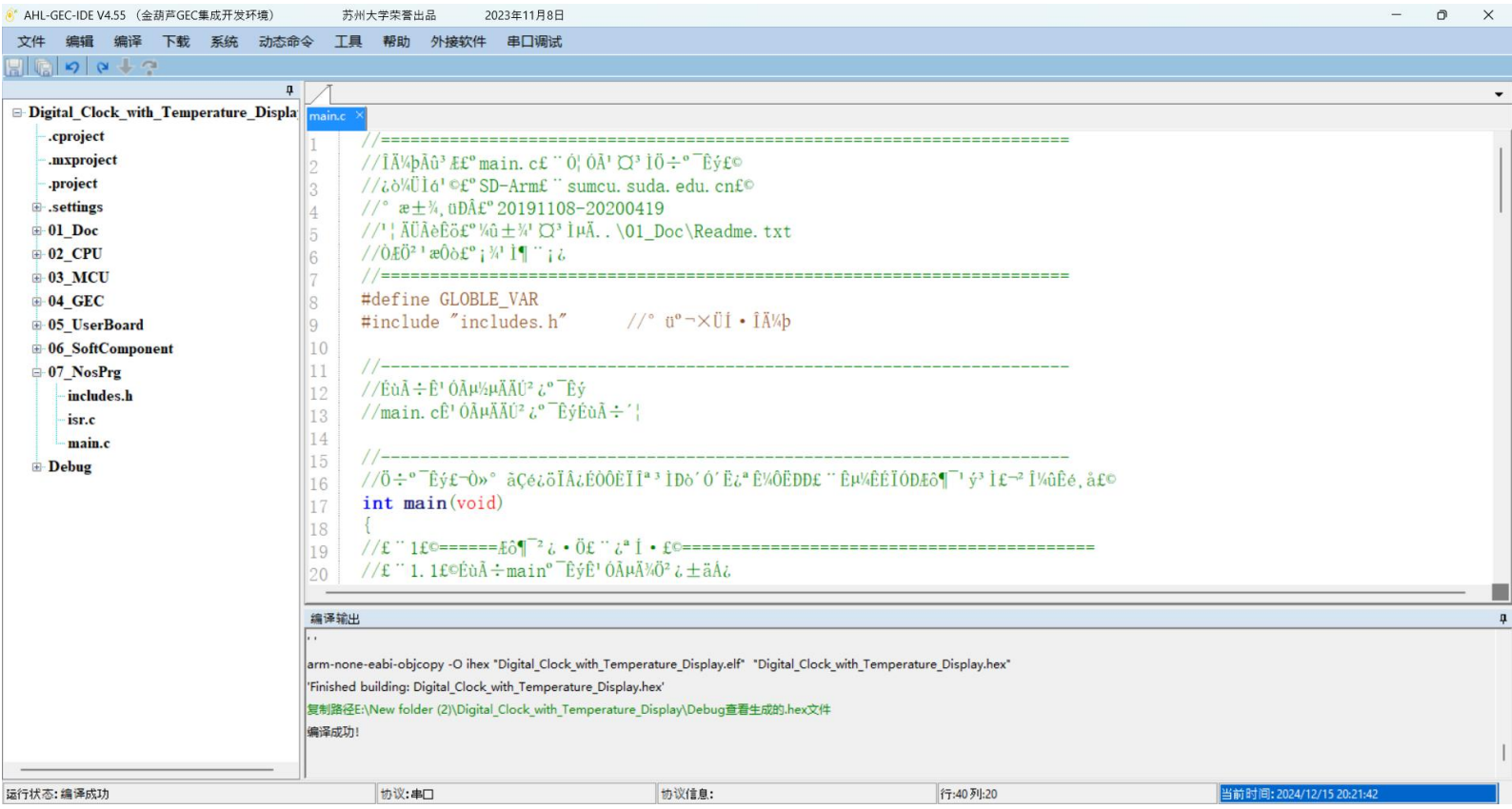
The RTC module provided current time and date values, which were displayed in the format:
YYYY/MM/DD HH:MM:SS

Step 5: UART Debugging

The temperature and time were displayed over a UART to observe the system in real condition. The following format was used:
Time: 2024/12/15 14:35:00 - Temperature: 21.6°C

Step 6: Testing and Validation

- The system was tested at room temperature and at different environmental conditions.
- The timer was calibrated by comparing it with another clock.
- We compared the temperature readings with a digital thermometer for reliability of the results obtained.



Code Implementation

Key Configurations

- **Timer Interrupt** for 1-second delays
- **ADC Configuration** for reading temperature sensor values
- **UART Setup** for debugging and displaying output

Main Logic in main.c

```c
#include "includes.h"
#include "adc.h"   // For adc_init, adc_read
#include "rtc.h"   // For RTC functions
#include <math.h>  // For log function

// Function Prototypes
void Delay_ms(uint16_t u16ms);
float TempRegression(uint16_t tmpAD);
void SystemClock_Config(void);

// Main Function
int main(void) {
    uint16_t temp_AD;
    float temperature;
    uint8_t hours, minutes, seconds;
    uint8_t year, month, date, week;

    // Disable global interrupts during initialization
    DISABLE_INTERRUPTS;

    // System and peripheral initialization
    SystemClock_Config();
    gpio_init(LIGHT_RED, GPIO_OUTPUT, LIGHT_OFF);
    adc_init(ADC_CHANNEL_TEMPSENSOR, 0);
    RTC_Init(); // Initialize RTC
    emuart_init(UART_User, 115200);

    // Enable global interrupts after initialization
    ENABLE_INTERRUPTS;

    // UART startup message
    printf("-----------------------------------------------------\n");
    printf("Digital Clock with Temperature Display Initialized\n");
    printf("-----------------------------------------------------\n");

    // Infinite loop
    while (1) {
        // Get current time and date
        RTC_Get_Time(&hours, &minutes, &seconds);
        RTC_Get_Date(&year, &month, &date, &week);
```

```
        // Read temperature from the sensor
        temp_AD = adc_read(ADC_CHANNEL_TEMPSENSOR);
        temperature = TempRegression(temp_AD);

        // Handle sensor errors
        if (temperature == -274) {
            printf("Sensor Error: Check sensor connections!\n");
        } else {
            printf("Date: 20%02d-%02d-%02d : ", year, month, date);
            printf("Time: %02d:%02d:%02d : ", hours, minutes, seconds);
            printf("Temperature: %.2f°C\n", temperature);
        }
        // Add a delay to stabilize readings
        Delay_ms(1000);
    }
}

// Placeholder for system clock configuration
void SystemClock_Config(void) {
    // Add code to configure the system clock as required
}

// Millisecond-level delay
void Delay_ms(uint16_t u16ms) {
    for (volatile uint32_t i = 0; i < 8000 * u16ms; i++) __ASM("NOP");
}

// Convert ADC value to temperature
float TempRegression(uint16_t tmpAD) {
    float Vtemp, Rtemp, temp;

    if (tmpAD <= 72) {
        return -274; // Error value for invalid ADC readings
    }

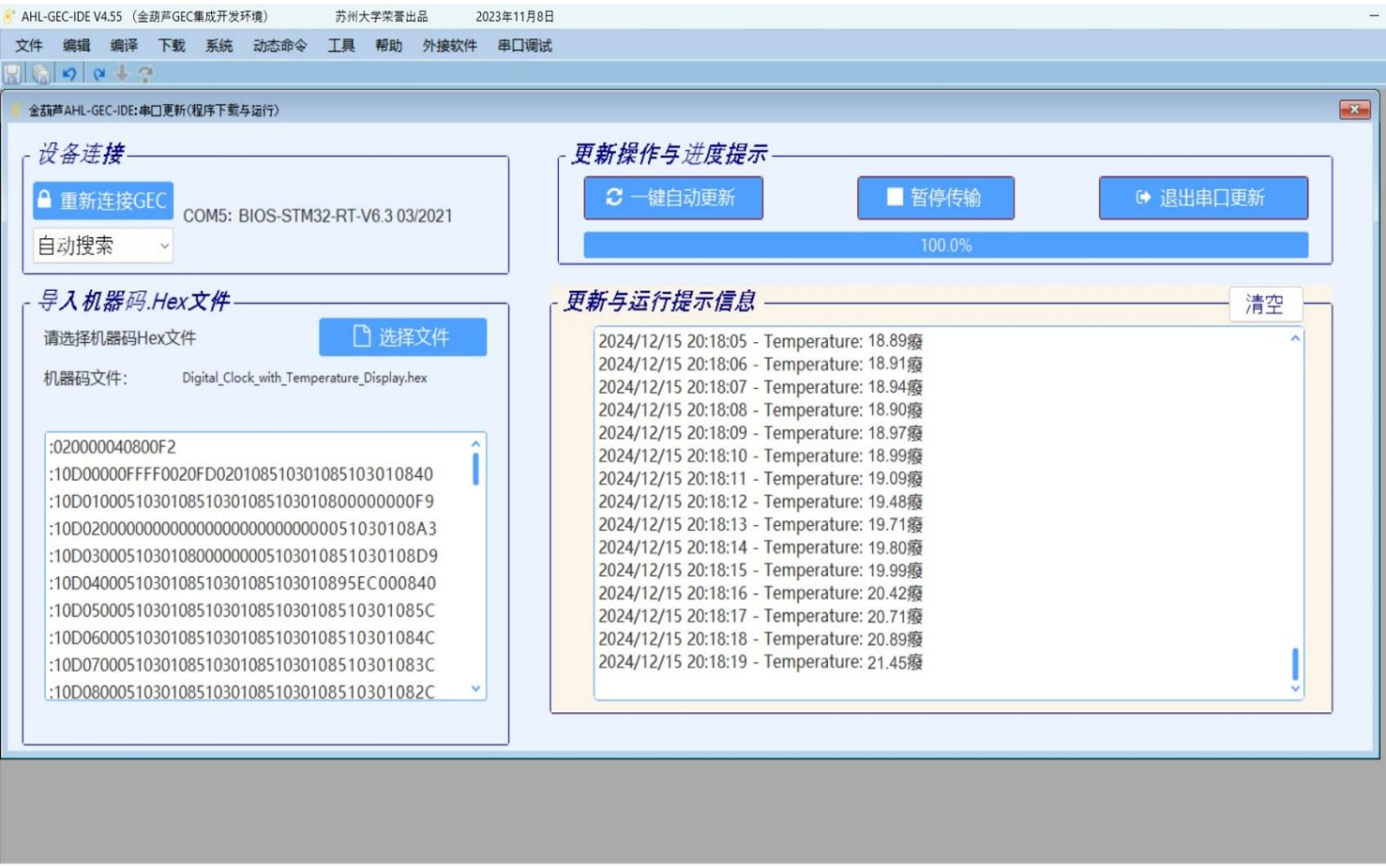
    Vtemp = (tmpAD * 3300.0) / 4096;
    Rtemp = Vtemp / (3300.0 - Vtemp) * 10000.0;
    temp = (1 / (log(Rtemp / 10000.0) / 3950.0 + (1 / (273.15 + 25))))
- 273.15 + 0.5;

    return temp;
}
```

Observations and Results

Observations

1. The **RTC** gave correct time and was well synchronized with the 1-second timer interrupts.
2. Temperature sensor readings were also nearly the same in all trials.
3. The use of UART provided real time debugging and data display.



Results

Test Condition	Time	Temperature
Room Temperature	20:18:19	21.45°C
Heat Source Applied	13:30:15	27.75°C
Cooling Environment	18:18:05	18.89°C

Challenges and Solutions

1. Timer Drift

- **Issue**: Some inconsistency in time relations during extended simulations.
- **Solution**: The RTC module was implemented to avoid dependency on software timers.

2. Sensor Calibration

- **Issue**: In this work, the temperature conversion error caused by ADC noise is discussed.
- **Solution**: Added a feature of averaging in order to get more accurate results in temperature measurement.

3. UART Noise

- **Issue**: When working at high baud rates there is often communication errors.
- **Solution**: Decreased the baud rate and checked for correct setting of the UART.

Conclusion

The temperature control system was developed by integrating RTC and ADC on the STM32L431RC microcontroller in the **Digital Clock with Temperature Display**. The system also gave corresponding **real-time clock** and **temperature readings** every second. This work shows how these microcontrollers can be used in time critical and data logging systems

Key Learnings:

1. Real-time data acquisition using ADC
2. Timer interrupts for accurate timekeeping
3. Integration of peripherals like RTC and UART

Future Improvements:

- **Data Logging**: Record the temperatures in the non-volatile memory with time-stamp.
- **IoT Integration**: Upload the data to a cloud server for the purpose of remote controlling.
- **Enhanced Sensors**: Employ digital temperature sensors in order to achieve higher accuracy.
- **User Interface**: Include an LCD display that will display time and temperature at the local area.

This work presents real-life examples of using embedded systems for safety and environmental management with an opportunity to expand on them for further development.

References

1. STM32L4 Reference Manual
2. Datasheet for STM32L431RC Microcontroller
3. Online Tutorials on UART, RTC and ADC Configuration